PROGREE REMOTE INTERNSHIP / FULL-STACK DEVELOPMENT

Internship work and project report

Four tasks completed across one React, Express, and SQLite project.

Omar Mohamed Aly

September 2026

PROJECT LINKS

LinkedIn announcement

lnkd.in/p/ezSQn6P2

Source repository

github.com/Frehshi/Progree-Full-stack-internship

LinkedIn announcement

PUBLISHED

The announcement is live. The final report records the public URL so the post can be checked directly instead of relying on a screenshot.

PUBLIC POST

lnkd.in/p/ezSQn6P2

Caption used

I'm excited to share that I've been accepted into the Progree Remote Internship Program as a Full-Stack Development Intern.

Over the coming month, I'll be strengthening my ability to design responsive and accessible interfaces, build reliable backend APIs, work with persistent databases, and implement secure authentication and checkout workflows. I'm especially looking forward to connecting these skills through complete, practical applications rather than isolated exercises.

Thank you to Progree for this opportunity. I'm ready to learn, build, document the process, and keep improving with every task.

#fullstack #webdev #javascript #progree

Why it belongs in this report

The post sets out the same scope shown in the pages that follow: responsive frontend work, a database-backed API, authenticated user data, and a server-controlled checkout flow.

Responsive portfolio

The home route is a hand-built React portfolio with project case studies, a skills section, an about section, and direct links to the working applications.

What I changed for real screens

- The two-column introduction and project grid collapse before the text becomes cramped.
- Navigation becomes a keyboard-friendly menu with an announced expanded state.
- Spacing, type size, and preview height are adjusted at three breakpoints rather than scaled uniformly.
- Visible focus states and reduced-motion support are included in the base styles.

Built with

React 19

Vite

Semantic HTML

CSS Grid and Flexbox

Custom responsive CSS

Checked

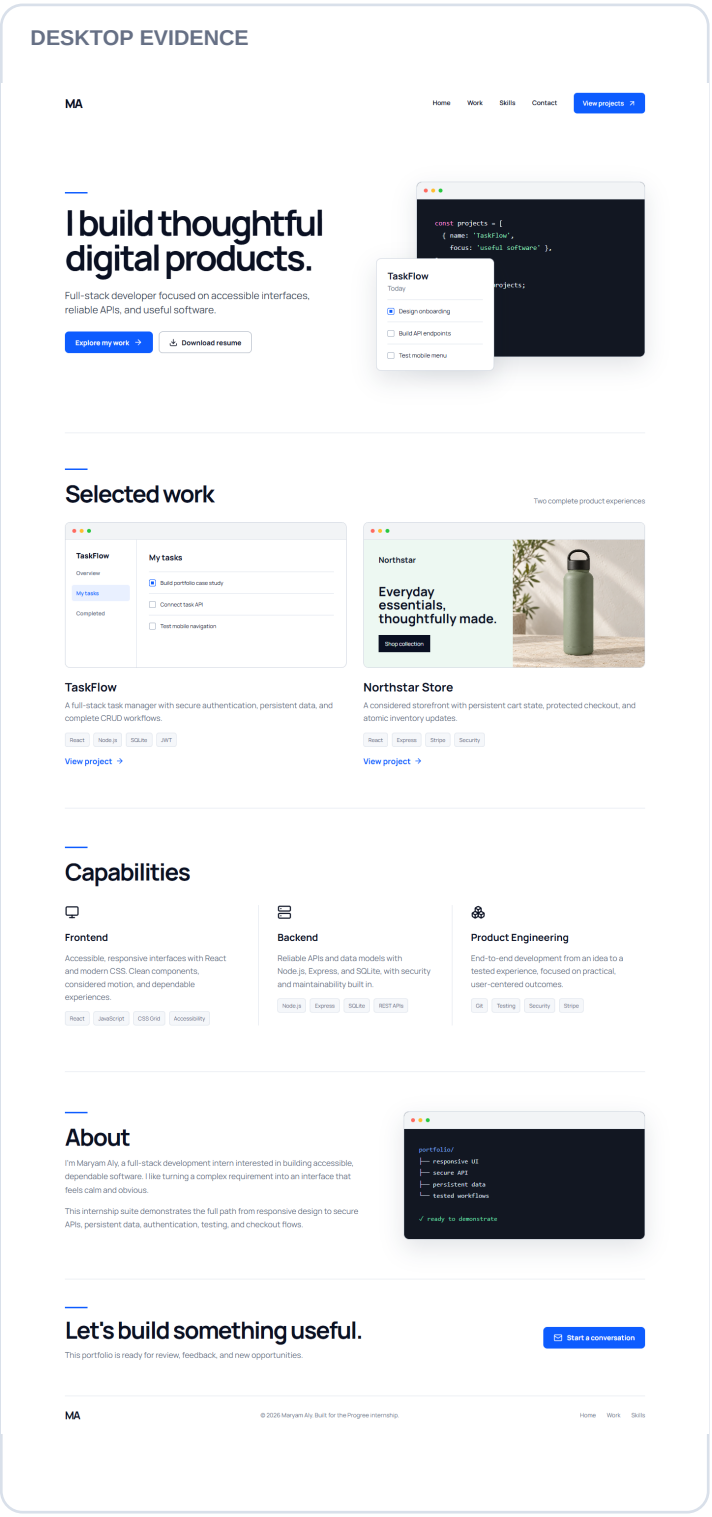
Production build

Component render test

1440 px desktop review

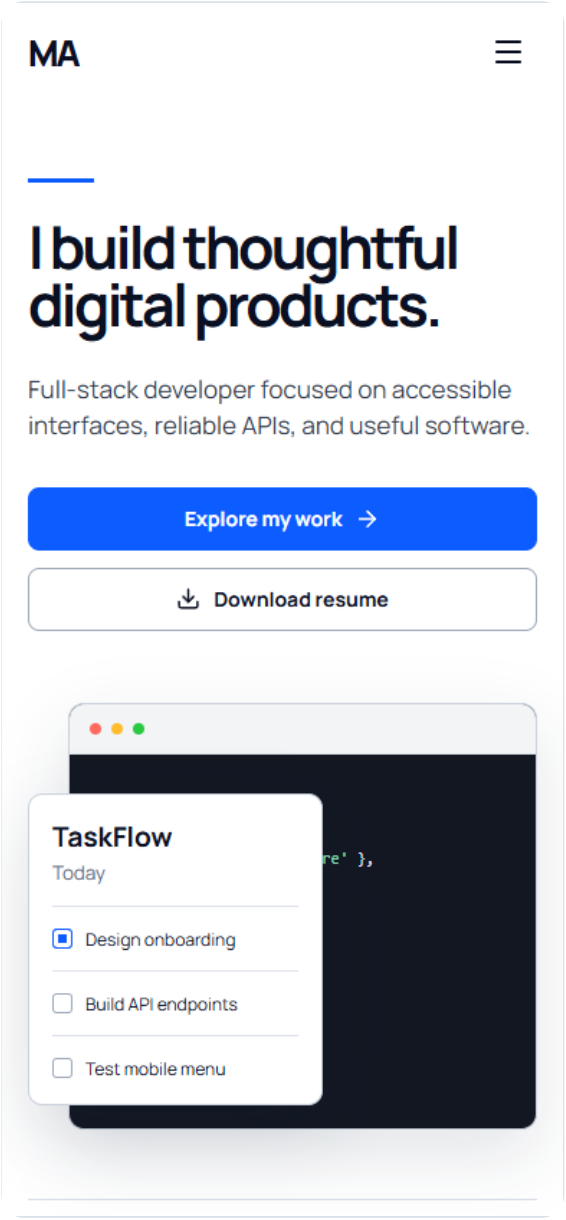
390 x 844 mobile review

DESKTOP EVIDENCE



Same content, different composition

The mobile page keeps the desktop hierarchy but changes the way it is read: navigation is compact, cards stack, and primary actions remain reachable without sideways scrolling.



01 Content order

The introduction and selected work stay first; supporting details move below them.

02 Touch targets

Menu and project actions keep generous hit areas and visible focus treatment.

03 No layout shortcut

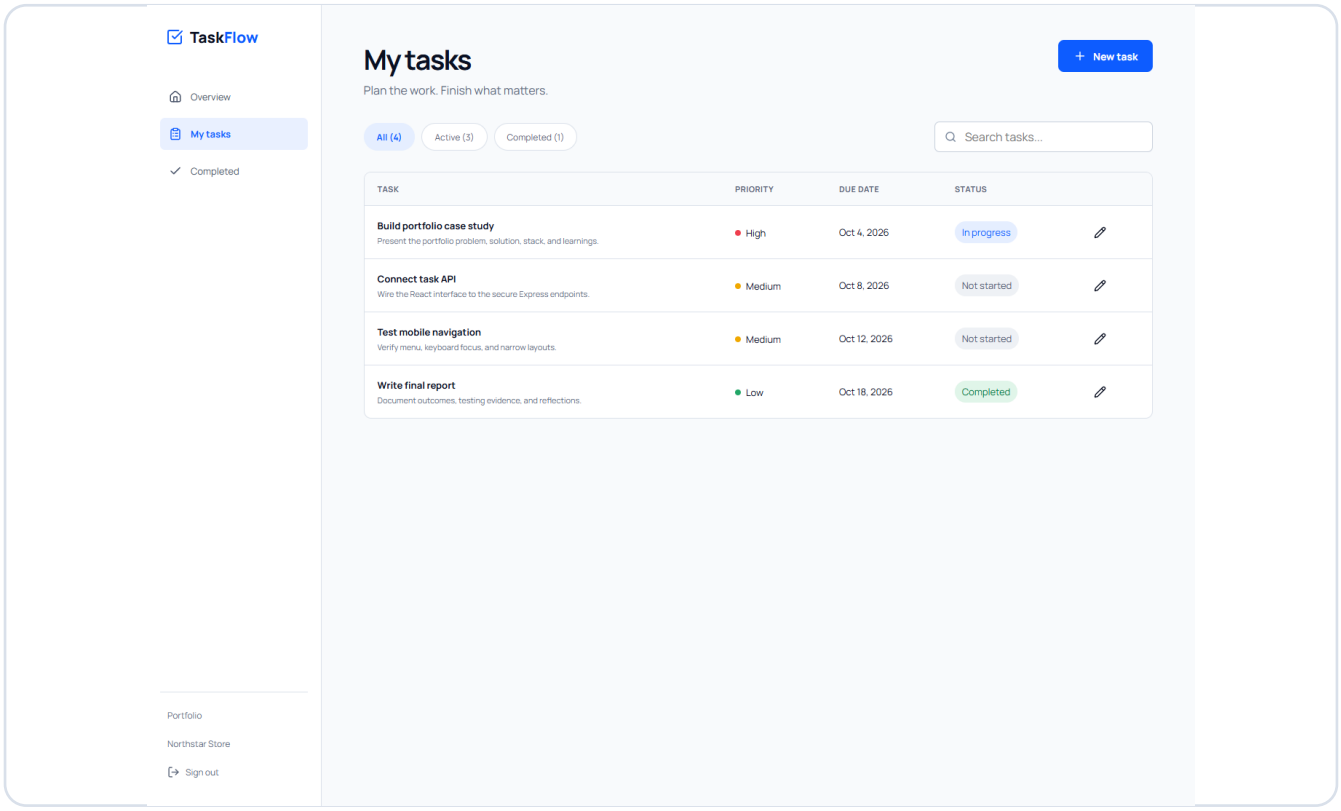
The page uses custom media queries; no generic responsive theme is doing the work.

04 Useful links

The resume, source repository, LinkedIn post, dashboard, and store are connected from the portfolio.

TaskFlow: authenticated CRUD

TaskFlow stores real user data in SQLite. After signing in, a user can create, read, edit, complete, search, filter, and delete tasks without leaving the page.



The request path

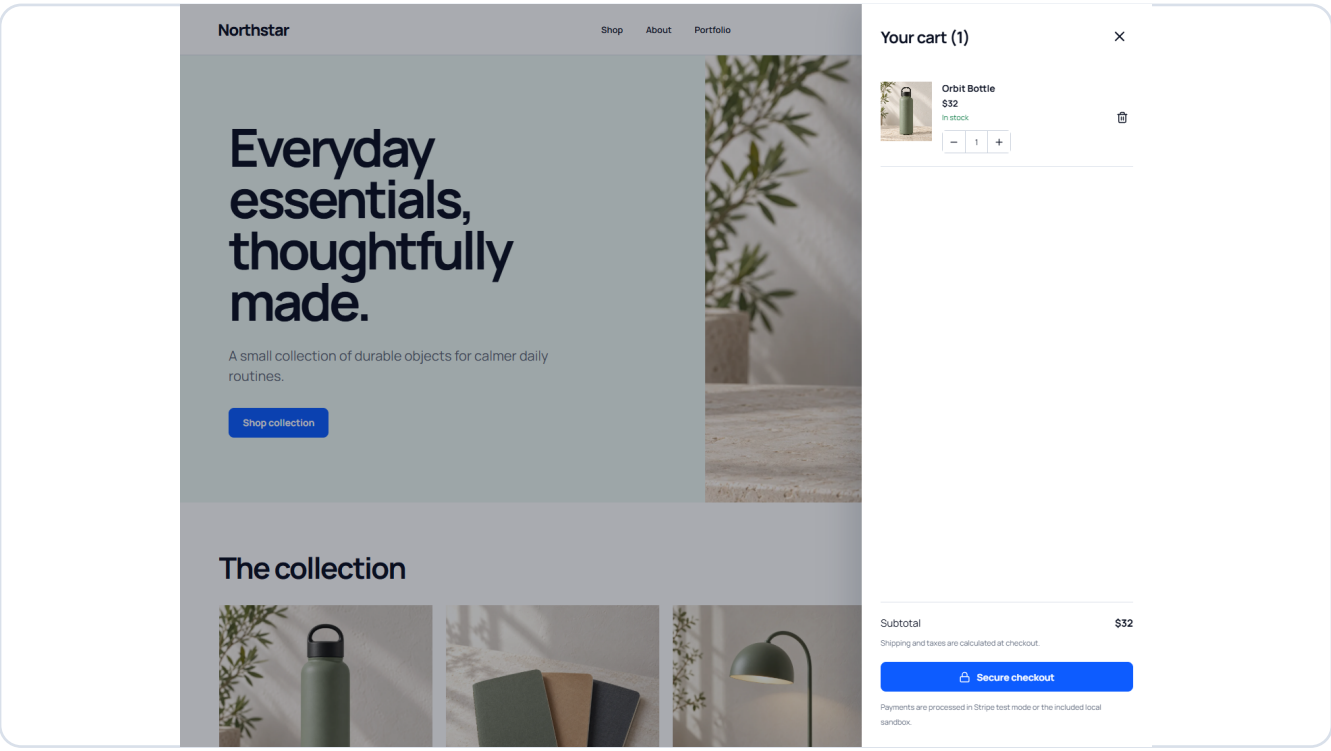
01 React form — **02** Express API — **03** JWT + Zod — **04** SQLite

- Passwords are hashed with bcrypt; access tokens expire after two hours.
- The user ID comes from the verified token, not from a request body.
- Every task query is scoped to its owner and every write is validated on the server.
- Integration tests cover the full create-edit-delete path and rejected unauthorised requests.

TASK 04

Northstar: secure test checkout

Northstar combines a product catalogue, authenticated cart, and server-controlled checkout. The local sandbox follows the same completion path as Stripe, so the project is reviewable without external credentials.



What the browser is allowed to do

Choose products, change quantities, and request checkout. The browser never supplies a trusted order total and cannot mark an order as paid.

What the server decides

Reload current prices and stock, create the order snapshot, verify the completion event, and update stock in one transaction.

PAYMENT COMPLETION RULE

A success page is not proof of payment.

Inventory changes only after a verified Stripe event or the trusted local sandbox routine. Event IDs are stored to prevent duplicate completion.

Decisions behind the interface

These are the parts that mattered most while building the three connected experiences.

One application, clear routes

The portfolio, task dashboard, and store share one React application and navigation, while each route keeps its own layout and interaction model.

Persistent local data

SQLite stores accounts, tasks, products, cart items, orders, order items, and processed checkout events. Refreshing the browser does not reset the workflow.

Server-side trust boundary

Prices, ownership, stock, and payment completion are checked on the API. Client-side controls improve usability but are not treated as security.

Review without secrets

A seeded demo account and local checkout sandbox let an evaluator use the complete project without a Stripe key or a separate database service.

Failure-aware writes

Checkout completion uses a database transaction. Insufficient stock rolls back the order update and cart changes together.

Evidence kept with the project

Desktop and mobile captures, editable task notes, the final PDF, and setup instructions live beside the source code.

Run, review, and submit

Start the project

```
npm install
copy .env.example .env
npm run dev
```

Review routes

Portfolio /

Task dashboard /tasks

Store /shop

Demo account

demo@progree.local

Demo123!

Verification completed

- Vite production build
- Three API integration tests and one frontend component test
- Desktop and 390 x 844 mobile browser review
- Production dependency audit with no known vulnerabilities

Repository and post

GITHUB REPOSITORY

github.com/Frehshi/Progree-Full-stack-internship/

LINKEDIN ANNOUNCEMENT

lnkd.in/p/ezSQn6P2

Submission contents

Four task reports, source code, generated assets, browser evidence, setup notes, and this combined PDF.